RealmOne

WIRESHARK LAB · NETWORK BASICS

Network Protocols

The APIs your computer already uses to talk to every other computer

> Hope you're ready for acronyms.

THE BIG IDEA

A protocol is an API between machines



When you call a library, you don't need to know how it works inside. You follow its contract: what to send, what comes back, and what the errors mean.

A protocol is the same kind of contract, written for two machines instead of two pieces of code. Every device follows the same spec, so any two can talk.

And it's already written. It ships in your OS, your browser, and libraries like requests.

IN YOUR CODE		ON THE NETWORK
Function signature what you pass in	→	**Message format** fields and headers every message needs
Call order open, read, close	→	**Message order** handshake, data, teardown
Return values and exceptions return x, raise ValueError	→	**Status and error codes** 200 OK, 404 Not Found
A library you import requests, socket	→	**Code built into your OS** already on every device

UNDER THE HOOD

One line of Python. Six protocols.

YOU WRITE THIS

```
requests.get("https://example.com")
```

AND YOUR COMPUTER DOES ALL OF THIS

01

DHCP

GET AN ADDRESS

Gave your laptop an IP address when it joined Wi-Fi

02

ARP

FIND THE ROUTER

Finds your router's MAC address so the first hop can be delivered

03

DNS

FIND THE SERVER

Turns example.com into an IP address

04

TCP

CONNECT

Opens a reliable connection with a three-way handshake

05

TLS

ENCRYPT

Encrypts the connection and checks the server's certificate

06

HTTP

ASK FOR THE PAGE

Sends GET / and gets back 200 OK

You wrote one line. Wireshark lets you watch every one of these happen.

LAYERS OF ABSTRACTION

The OSI model: 7 layers of abstraction

7	**Application**	Protocols apps use to talk to each other	`HTTP, DNS, FTP`
6	**Presentation**	Data translation: encoding, compression, encryption	`TLS, UTF-8`
5	**Session**	Opening, managing, and closing conversations	`RPC, NetBIOS`
4	**Transport**	Process-to-process delivery, using ports	`TCP, UDP`
3	**Network**	Addressing and routing across networks	`IPv4, IPv6`
2	**Data Link**	Delivery between devices on the same network	`Ethernet, Wi-Fi, ARP`
1	**Physical**	Bits as electrical, light, or radio signals	`cables, fiber, radio`

WHY LAYERS?

Each layer only calls the layer directly below it, through a fixed interface. HTTP doesn't care whether it's riding on Wi-Fi or fiber, the same way your code doesn't care which disk a file is on.

IN PRACTICE

The internet runs on the simpler 4-layer TCP/IP model, where layers 5 to 7 are one application layer. OSI is the shared vocabulary: "that's a layer 2 problem."

ENCAPSULATION

Every layer wraps the one above it

ETHERNET FRAME adds source and destination MAC address

IP PACKET adds source and destination IP address

TCP SEGMENT adds ports, sequence numbers, ACKs

HTTP REQUEST the data you actually sent

```
GET / HTTP/1.1
Host: example.com
```

SCAPY BUILDS PACKETS THE SAME WAY

```
from scapy.all import Ether, IP, TCP
pkt = (Ether()
       / IP(dst="128.119.245.12")
       / TCP(dport=80)
       / "GET / HTTP/1.1\r\n\r\n")
pkt.summary()
# 'Ether / IP / TCP ... / Raw'
```

IN WIRESHARK

The middle pane is this diagram, outside in: Ethernet II, Internet Protocol, TCP, then HTTP. Expand each one to see its header fields.

RealmOne

THE API DOCS

Protocol by protocol

What each one does, how your code already calls it, and how to find it in Wireshark

> Real code is Python you can run. Pseudo-code shows the idea.

LAYER 3 · NETWORK

IP: Internet Protocol

Gives every device an address and moves packets toward that address, one router hop at a time.

cough* hence the name IP address *cough

- Best effort: packets can be dropped, duplicated, or arrive out of order. Fixing that is TCP's job.
- IPv4 addresses look like `128.119.245.12`; IPv6 like `2001:db8::1`.
- Every router lowers the TTL by one and drops the packet at zero, so lost packets can't loop forever.
- Try it: `traceroute gaia.cs.umass.edu` (Windows: `tracert`) lists every router on the way.

API · PSEUDO-CODE

```
# TCP and UDP call this for you
ip.send(
    src="192.168.1.23",
    dst="128.119.245.12",
    ttl=64,
    payload=segment,
)
# returns nothing, promises nothing
```

FIND IT IN WIRESHARK

`ip.addr == 128.119.245.12`

Anything to or from gaia, the lab's web server

LAYER 4 · TRANSPORT

TCP: Transmission Control Protocol

Turns IP's unreliable packets into a reliable, ordered stream of bytes between two programs. The port number says which program.

- Opens with a three-way handshake: SYN, SYN-ACK, ACK.
- Numbers every byte. Anything the other side doesn't acknowledge gets resent.
- Closes with FIN and ACK.
- Used by HTTPS, SSH, email, and file transfers.

REAL CODE · PYTHON

```
import socket
s = socket.create_connection(("gaia.cs.umass.edu", 80))
# the handshake happens inside that call
s.sendall(b"GET / HTTP/1.0\r\n\r\n")
data = s.recv(4096)  # in order, no gaps
s.close()            # FIN, ACK
```

FIND IT IN WIRESHARK

`tcp.flags.syn == 1`

The opening SYN and SYN-ACK of every handshake

LAYER 4 · TRANSPORT

UDP: User Datagram Protocol

Sends independent messages (datagrams) to a port with no setup and no promises. Less overhead means lower latency.

- No handshake, no acknowledgments, no retransmits.
- Messages can be lost or arrive out of order. The app decides whether it cares.
- Great when late data is useless anyway: video calls, games, live streams.
- Also carries DNS, DHCP, and QUIC (HTTP/3).

REAL CODE · PYTHON

```python
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.sendto(b"hello?", ("192.168.1.50", 9999))
# returns 6 (bytes sent), and that's it:
# no connection, no reply, no idea if it arrived
```

FIND IT IN WIRESHARK

udp

DNS, QUIC, mDNS, and more

ARP: Address Resolution Protocol

IP addresses get a packet to the right network. MAC addresses get it to the right device on that network. ARP finds the MAC address that goes with an IP.

- Wireshark shows it as `Who has 192.168.1.1? Tell 192.168.1.23`
- The question is broadcast to every device on the network. Only the owner answers.
- Answers are cached, so your computer doesn't ask every time.
- Try it: `arp -a` shows your computer's cache.

API · PSEUDO-CODE

```
# your OS calls this before talking to the router
mac = arp.who_has("192.168.1.1")

# on the wire:
#   "Who has 192.168.1.1? Tell 192.168.1.23"
#   "192.168.1.1 is at 02:00:00:00:00:fe"
```

FIND IT IN WIRESHARK

`arp`

Mostly broadcasts, from every device on your network

LAYER 7 · APPLICATION

DHCP: Dynamic Host Configuration Protocol

When you join a network, DHCP hands your device everything it needs to use it: an IP address, the router's address, and which DNS server to ask.

- Four messages: Discover, Offer, Request, Acknowledge (DORA).
- Runs over UDP: port 67 on the server, 68 on the client.
- Addresses are leased, not owned. Your device renews before the lease runs out.
- Try it: `ipconfig getpacket en0` (Mac) or `ipconfig /all` (Windows).

API · PSEUDO-CODE

```
# your OS calls this when you join Wi-Fi
lease = dhcp.request_lease()   # DORA

lease.ip           # "192.168.1.23"
lease.router       # "192.168.1.1"
lease.dns_servers  # ["192.168.1.1"]
lease.lease_time   # 3600 (seconds)
```

FIND IT IN WIRESHARK

dhcp

Turn your Wi-Fi off and on while capturing

LAYER 7 · APPLICATION

DNS: Domain Name System

The internet's phone book. Turns names people can remember into IP addresses computers can route to.

- Usually UDP port 53 and unencrypted, so anyone on the path can see which names you look up.
- Answers are cached, each with a time to live (TTL).
- Record types: A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail).
- Cloudflare's 1.1.1.1 is a great alternative to your ISP's resolver. Or build a Pi-hole.

REAL CODE · PYTHON + TERMINAL

```
import socket
socket.gethostbyname("gaia.cs.umass.edu")
# '128.119.245.12'

# or from any terminal (Mac, Linux, Windows):
nslookup gaia.cs.umass.edu
```

FIND IT IN WIRESHARK

dns

Every name your computer looked up (unless your browser uses DNS over HTTPS)

LAYER 7 · APPLICATION

HTTP: Hypertext Transfer Protocol

The request/response protocol of the web, on TCP port 80. The client sends a method and a path. The server sends back a status code, headers, and a body.

METHODS: THE CRUD OF THE WEB

GET	read
POST	create
PUT	replace
DELETE	delete

ON THE WIRE, IT'S JUST TEXT

```
GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1
Host: gaia.cs.umass.edu

HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
```

REAL CODE · PYTHON

```python
import requests
base = "http://gaia.cs.umass.edu/wireshark-labs/"
r = requests.get(base + "INTRO-wireshark-file1.html")
r.status_code               # 200
r.headers["Content-Type"]   # 'text/html; charset=UTF-8'
r.text                      # the page itself
```

FIND IT IN WIRESHARK

`http`

Expect very little: almost every site uses HTTPS now

LAYER 7 · APPLICATION

Status codes are HTTP's return values

Every response starts with a status line like `HTTP/1.1 200 OK`. The first digit is the category.

1xx

Informational

Hold on, there's more

`101 Switching Protocols`

2xx

Success

It worked

`200 OK`

3xx

Redirection

It's over there

`301 Moved Permanently`

4xx

Client error

You messed up

`404 Not Found`
`401 Unauthorized`

5xx

Server error

We messed up

`500 Internal Server Error`

IN CODE, 4XX AND 5XX BECOME EXCEPTIONS

```
r = requests.get(url)
r.raise_for_status()  # raises HTTPError on any 4xx or 5xx
```

IN THE LAB

The server answers your first request with 401 Unauthorized. Watch what your client sends next.

TRANSPORT LAYER SECURITY

HTTPS: HTTP inside TLS

TLS wraps a TCP connection in encryption and proves the server is who it says it is. HTTPS is plain HTTP sent through TLS, on port 443.

- Almost all web traffic you capture today is TLS or QUIC. Wireshark sees that you talked, not what you said.
- It can still see the server name your computer sends first (SNI), unless the site uses Encrypted Client Hello.
- HTTP/2 (2015) speeds up HTTPS. HTTP/3 (2022) runs over QUIC, on UDP port 443.

REAL CODE · PYTHON

```python
import socket, ssl
ctx = ssl.create_default_context()
raw = socket.create_connection(("example.com", 443))
s = ctx.wrap_socket(raw, server_hostname="example.com")
s.version()   # 'TLSv1.3'
# server_hostname (the SNI) is sent unencrypted
```

FIND IT IN WIRESHARK

`tls.handshake.extensions_server_name`

The sites you visited, even over HTTPS

LAYER 7 · APPLICATION

FTP: File Transfer Protocol

Moves files between a client and a server (imagine that), on ports 20 and 21. It was designed long before anyone worried about eavesdroppers.

- Your username and password cross the network as plain text.
- SFTP (SSH File Transfer Protocol, port 22) is the secure replacement.
- TFTP (Trivial FTP, UDP port 69) moves small files with no login at all.

REAL CODE · PYTHON (EXAMPLE SERVER)

```
from ftplib import FTP
ftp = FTP("ftp.example.com")
ftp.login("alice", "hunter2")

# what actually crosses the wire:
#   USER alice
#   PASS hunter2
```

FIND IT IN WIRESHARK

`ftp`

TCP port 21: look for the USER and PASS commands

BONUS PROTOCOL

Telnet: remote login, in plain text

Before SSH, telnet was how you logged in to another computer's terminal. Everything you type, passwords included, crosses the network as plain text.

- TCP port 23.
- SSH (Secure Shell, port 22) is basically secure telnet.
- Capture it, then right-click a packet and choose Follow, then TCP Stream, to read the whole conversation.

TRY IT · TERMINAL

```
# Mac / Linux
nc towel.blinkenlights.nl 23

# Windows: turn on "Telnet Client" in
# Windows Features first
telnet towel.blinkenlights.nl
```

FIND IT IN WIRESHARK

`tcp.port == 23`

Then right-click a packet: Follow, then TCP Stream

BONUS PROTOCOLS AGAIN

Two more you'll see in every capture (last slide I promise)

QUIC

UDP PORT 443

The encrypted transport under HTTP/3. It rebuilds TCP's reliability and TLS's encryption on top of UDP, so connections start faster.

Filter `quic`

mDNS

UDP PORT 5353

Multicast DNS: devices announcing themselves on your local network, like printers and AirPlay speakers. It's how your laptop finds the printer without anyone typing an IP address.

Filter `mdns`

RealmOne

UP NEXT

Time 4 Lab

github.com/ToofastTurnip/Wireshark-Lab

> Only capture on networks you own or have permission to monitor.